

SOURCE-A

AI Operating System

Cloud Kernel Architecture — Target Spec

Version 1.3 — Target Architecture · Phase 1 Partial

The deterministic cloud kernel for a multi-tenant factory runtime. This document is the **target vision** — the live runtime reality is the L0 Mac Workbench. Truth is reconciled through the SSOT Index.

Contracts run the system — not LLMs

Blueprint Registry

Receipts > Diagrams

9.3/10 External review · production-grade thinking

Noetfield Systems Inc. | TrustField Technologies Inc.

Vancouver, BC — 2026 · Status: **TARGET / PHASE 1 PARTIAL**



Read this first — what this document is, and is not

This is the **Target Kernel (L1–L8)**, not current production. Several layers here are design, not deployed. The system that runs today is the **LO Mac Workbench** (Hub, AG Routing, Mac Law, the SCAN → SHIP belt, on-disk receipts) on a Railway FBE motor. The two layers stack — they do not compete. Authority over which is which lives in the [SOURCEA_SSOT_INDEX](#) .

Foundation & Truth Model

How truth is managed, and the principles that run the kernel

1

The Truth Model

NEW IN v1.3

The hardest problem was never the architecture — it was **truth management**. Multiple valid documents each claimed to be “THE SSOT,” which made agents build on false assumptions. The fix is a single Truth Index with three clearly-scoped tiers.

SSOT Index

The single entry point every agent reads first. Points to the two scopes below and the path between them.

L0 · Runtime Reality

What exists on disk and on the server *today*: Mac Workbench, Railway FBE, Hub, on-disk receipts. Shipped and running.

L1–L8 · Target Kernel

What we are building toward: this document — PEVC loop, Execution Contracts, the unified data graph, the router.

Migration Path

The honest gap between Reality and Target, layer by layer, with a status on each (today → later).

The one principle to keep if you keep nothing else

Fundable systems are proven by receipts, not diagrams. Investors do not fund Neon, Cloudflare Workers, an 8-layer chart, or a PEVC drawing. They fund a factory that actually ran: *Input → Artifact → Evidence → Decision → Receipt*. One auditable vertical slice outweighs a hundred pages of architecture.

2

Architectural Philosophy — The 4 Pillars

Specialization occurs in the **configuration layer** (Data Graph / Blueprints), **not in the infrastructure layer** (Workers / Code).

~~LLMs do not run the system.~~

Contracts run the system.

LLMs only propose plans.

Pillar	Name	Responsibility
P1	Product Layer UI / Config	Business skins and front-ends (Forge, Noetfield, TrustField, WitnessBC, 777). Zero custom infrastructure — only presentation and config.
P2	Governance Layer Policy / SLA	Quality control, validation loops, compliance, and SLA definitions. Defines the unyielding rules of execution.
P3	Runtime Kernel Stateless Workers	Stateless, memoryless workers. Blind execution muscle taking structured commands from data.
P4	Factory Data Graph Supabase / Neon	The connected multi-tenant ecosystem, orchestrated entirely via the Data Graph.

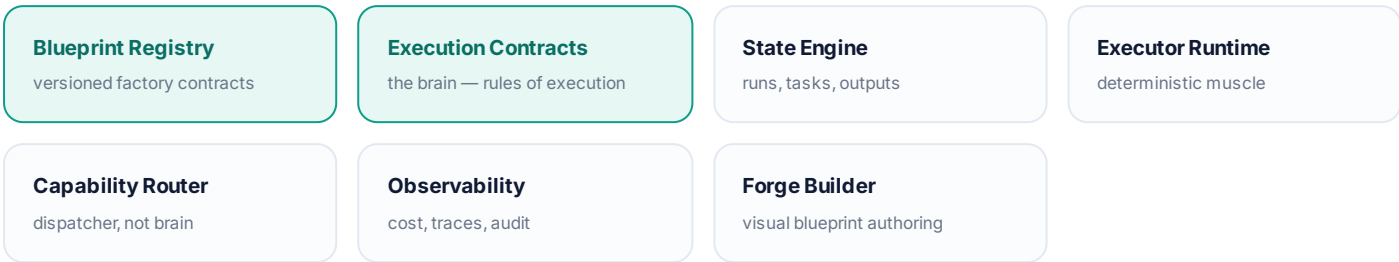
The Golden Rule

Workers are muscle, never the brain. The brain is the **Execution Contract** stored in the Data Graph. The LLM proposes; the Contract decides what is allowed, what output is valid, what cost is permitted, and what rollback is required. The Kernel holds absolute Execution Authority.

3 The Source-A Kernel — Component Map

As of v1.3 the kernel is understood as seven components, with the Execution Contract — not the Router — at the center.

Source-A Kernel



4 The 8-Layer Kernel — with Honest Status

The decoupled 8-layer stack, each layer labeled with what runs *today* versus the *target*. No layer is described as more mature than it is.

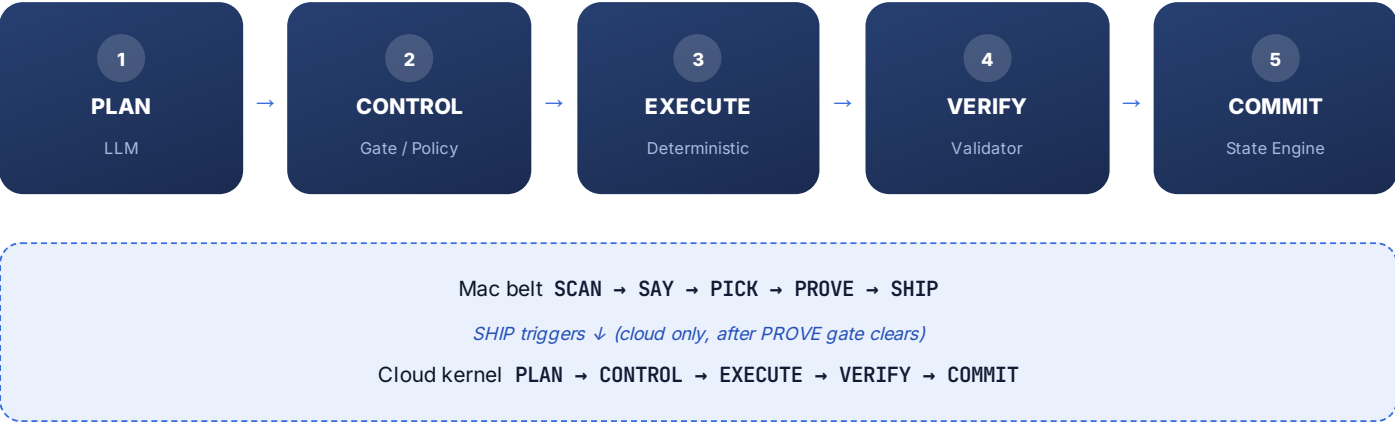
Layer	Component	Target Technology	Status today
L1	UI / Interface	Next.js + CF Pages	Vercel + static Hub HTML
L2	Auth & Isolation	Supabase Auth + RLS	Live
L3	State Engine	Neon (target) / Supabase (now)	Supabase until ASF-approved Neon migration
L4	Queue Engine	CloudFlare Queues	Hub drain + phase-observed JSON
L5	Runtime Engine	4 stateless workers	Contract-gated Railway FBE
L6	Capability Router	Dynamic Tool Registry	OpenRouter + Cursor pools
L7	Heavy Compute	Modal / RunPod	Fal · ElevenLabs · video-ad-factory
L8	Observability	OpenTelemetry + ClickHouse	JSON receipts + Hub logs only

L0 — the missing layer. Beneath all of the above sits the Mac Hardened Machine Workbench (Hub :13020, AG Routing :8782, Mac Law :8781, the SCAN → SAY → PICK → PROVE → SHIP belt). It is the control plane and gate. See `SOURCEA_HARDENED_MACHINE_WORKBENCH_ARCHITECTURE_LOCKED_v1` .

5

The Controlled Runtime Machine — 5-Stage Loop

Every computational step runs through a strict pipeline. Without it the system is probabilistic execution; with it, a controlled runtime machine. In production this loop nests *inside* the Mac belt's SHIP step — it fires only after the founder gate (PROVE) clears.



- 1

Plan (Suggestion Engine). The Planner uses an LLM to break task intent into a structured DAG. Output is structured JSON only — never free-form action.
- 2

Control (Gate / Policy). Before execution, input is validated against the contract schema; permissions, tool allowlist, and SLA cost-guards are enforced. On failure: reject or rewrite.

- 3 **Execute (Deterministic Muscle).** The Executor triggers the exact programmatic call. Zero creativity, zero prompt alteration, zero model guessing — only `tool(input) → output` .

- 4 **Verify (Verification Loop).** Output is checked against validation contracts (size, schema, token limits). On failure: exponential-backoff retry or escalation.

- 5 **Commit (State Engine).** Verified state is finalized: asset URLs as *Artifacts*, reasoning trails as *Evidence*, outcomes as *Decisions*, with a realtime signal to L1.

The Kernel — Contracts, Registry, Data

The brain of the system and the structures that keep it controllable

6 Execution Contract — The Brain

The Execution Contract — not the Router — is the brain. A correct contract constrains *which tools are allowed, which output is valid, which cost is permitted, and which rollback is required*. When the contract is right, the underlying model (Gemini, Claude, DeepSeek, Grok, or a local model) becomes fully swappable.

6.1 Plan Contract (LLM Output Schema)

The LLM may emit *only* this shape:

```
{
  "steps": [
    {
      "step_id": 1,
      "type": "tool_call",
      "tool": "llm | elevenlabs | ffmpeg | supabase",
      "action": "string",
      "input": {},
      "expected_output": "artifact | text | file | json",
      "sla": "draft | normal | premium",
      "idempotency_key": "uuid",          // MANDATORY in v1.3
      "depends_on": [],
      "compensation": {                 // NEW in v1.3 – rollback action
        "tool": "string",
        "action": "string",
        "input": {}
      }
    }
  ]
}
```

6.2 Control Layer — Hard Gate

✓ Schema Validation

- `steps` must be an array
- each `tool` exists in the registry
- no undefined actions

✓ Safety Rules

- no unauthorized external calls
- no filesystem access outside the sandbox
- no recursive self-modifying plans

✓ Cost Check

- if `estimated_cost > budget` → reject or downgrade tier

✓ Idempotency & DAG

- every step carries an `idempotency_key`
- the DAG is acyclic, no missing refs

6.3 Rollback & Compensation NEW IN v1.3

Critical steps — especially external API calls with side effects — must declare a **compensation action** that the kernel runs to undo partial work when a downstream step fails. This makes multi-step runs safely reversible rather than leaving orphaned artifacts.

```
// On failure of step N, the kernel walks completed steps in reverse
// and executes each declared compensation action before marking FAILED.
{
  "step_id": 2,
  "tool": "elevenlabs", "action": "tts", "input": { "...": "..." },
  "compensation": { "tool": "elevenlabs", "action": "delete_asset", "input": { "asset_id": "$ref:step_2.output.id" } }
}
```

6.4 Verification & Commit

On verify PASS

- ✓ `runs.status = success`
- ✓ artifacts / evidence / decisions saved
- ✓ realtime signal to L1

On verify FAIL

- ✗ retry (max 3, backoff + jitter)
- ✗ run declared compensations in reverse
- ✗ `runs.status = failed` / escalate

7 Blueprint Registry NEW IN v1.3

The biggest scaling risk is no longer the database or the cloud — it is **Blueprint Explosion**. As factories grow from 10 to 50 to 500 to 5,000 blueprints, their versions, dependencies, guards, and contracts become uncontrollable without a registry to govern them.

10 → 50 → 500 → 5,000 without a registry: uncontrollable

The registry adds four governing tables so every blueprint is versioned, dependency-aware, tested, and lifecycle-tracked:


```

-- Immutable version history per blueprint
CREATE TABLE blueprint_versions (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  blueprint_id UUID NOT NULL REFERENCES blueprints(id) ON DELETE CASCADE,
  version     INT NOT NULL,
  spec        JSONB NOT NULL,          -- frozen contract at this version
  created_by  UUID,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(blueprint_id, version)
);

-- Explicit dependency graph between blueprints (acyclic, enforced)
CREATE TABLE blueprint_dependencies (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  blueprint_id UUID NOT NULL REFERENCES blueprints(id) ON DELETE CASCADE,
  depends_on_id UUID NOT NULL REFERENCES blueprints(id),
  depends_on_version INT NOT NULL,
  UNIQUE(blueprint_id, depends_on_id)
);

-- Test fixtures + last result, gating promotion to active
CREATE TABLE blueprint_tests (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  blueprint_id UUID NOT NULL REFERENCES blueprints(id) ON DELETE CASCADE,
  fixture      JSONB NOT NULL,
  last_result  VARCHAR(20),           -- pass | fail | stale
  last_run_at  TIMESTAMPTZ
);

-- Lifecycle status - nothing runs in prod unless 'active'
CREATE TABLE blueprint_status (
  blueprint_id UUID PRIMARY KEY REFERENCES blueprints(id) ON DELETE CASCADE,
  status       VARCHAR(20) NOT NULL DEFAULT 'draft',
              -- draft | testing | active | deprecated | frozen
  updated_at   TIMESTAMPTZ DEFAULT NOW()
);

```

Registry table	Controls
blueprint_versions	Immutable history; every change is a new frozen version, never an edit in place.
blueprint_dependencies	An acyclic graph so a change in one blueprint surfaces every blueprint it affects.
blueprint_tests	Fixtures + last result; a blueprint cannot be promoted to <code>active</code> with failing tests.
blueprint_status	Lifecycle gate; only <code>active</code> blueprints execute in production.

8 Data Hierarchy & Blueprint Contract

```

Tenant (Organization)
├── Project (Brand: Noetfield | Forge | TrustField | WitnessBC | 777)
│   ├── Factory (Pipeline: Board Review | Risk Scan | Ad Generator)
│   │   ├── Blueprint (Immutable, versioned runtime contract) ── governed by
│   │   │   ├── Run (Live execution instance) ── Blueprint Registry
│   │   │   │   ├── Task (Atomic node, mandatory idempotency_key)
│   │   │   │   │   ├── Artifact (video | audio | document)
│   │   │   │   │   ├── Evidence (verification | proof)
│   │   │   │   │   └── Decision (approved | rejected | escalated)

```

Supabase — today

Auth, RLS, realtime, and the live state tier. This is the real database now.

Neon — target

Primary high-write State Engine *only after an ASF-approved migration*. Agents must not assume Neon exists yet.

9.1 Tasks — Idempotency Now Mandatory

```
CREATE TABLE tasks (
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  run_id            UUID NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  node_id           VARCHAR(100) NOT NULL,
  capability_required VARCHAR(150) NOT NULL,
  status            VARCHAR(50) NOT NULL DEFAULT 'pending',
  input_data        JSONB DEFAULT '{}':jsonb,
  output_data       JSONB DEFAULT '{}':jsonb,
  idempotency_key    VARCHAR(255) NOT NULL UNIQUE,      -- v1.3: NOT NULL, mandatory
  compensation       JSONB DEFAULT '{}':jsonb,         -- v1.3: rollback action
  retry_count        INT DEFAULT 0,
  started_at        TIMESTAMPTZ,
  completed_at       TIMESTAMPTZ,
  created_at         TIMESTAMPTZ DEFAULT NOW()
);
```

The full identity / execution / output / control / telemetry DDL and all performance indexes carry over unchanged from v1.2 §6. Only the deltas above (mandatory `idempotency_key`, `compensation`, and the four Blueprint Registry tables) are new in v1.3.

Runtime, Routing & Growth

Dispatch, Forge, and the path to a fundable demo

10 Capability Router — The Dispatcher

The Router is reclassified in v1.3: it is a **dispatcher, not the brain**. The contract already decided what is allowed; the Router only picks a provider for a capability + SLA, and protects the system with a circuit breaker. Models are interchangeable behind it.

80–85% · High Volume

Gemini Flash-Lite
DeepSeek V4 Flash

Intake, validation, logging, structured inputs — minimal cost.
Live today via OpenRouter + Cursor pools.

15–20% · Advanced Logic

Claude Sonnet
Grok 4.1 Fast

Governance, Forge code generation, multi-layer planning, final audits.

10.1 Circuit Breaker EXPLICIT IN v1.3

Resilience is now a named requirement of the Router, not an implementation detail: after N consecutive failures a provider is tripped open, traffic fails over to the declared fallback, and the breaker half-opens on a timer to test recovery.

```
async function resolve({ capability, sla }) {
  const config = await db.getCapabilityConfig(capability); // from Tool Registry
  const provider = selectProvider(config, sla);
  if (circuitBreaker.isOpen(provider)) { // tripped after N failures
    return config.fallback ?? escalate(capability);
  }
  return provider; // half-open probe restores it
}
```

11 The 4 Stateless Workers (Logical Roles)

Four immutable worker roles power the entire multi-product runtime. Today these roles are served by the contract-gated Railway FBE motor; the names below are logical, not a claim that four separate Cloudflare Workers are deployed.



W1

Intake

- Validates input against `inputs_schema`
- Writes a `runs` record (`pending`)
- Offloads `run_id` to the queue

W2

Planner

- Reads `workflow_graph` from the active blueprint version
- Emits a structured plan bound to an Execution Contract
- Orders dependencies, flags capabilities

W3

Executor

- Asks the Router for a provider
- Validates against the `execution_contract`
- Runs the call; records artifact / evidence / decision

W4

Memory

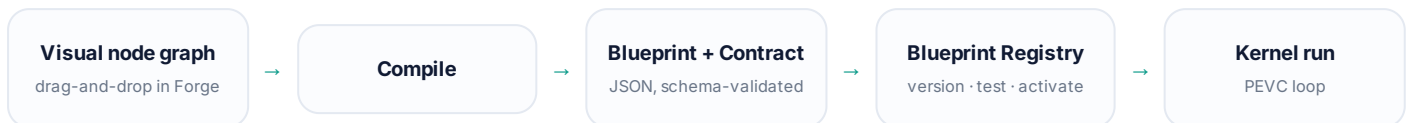
- Uploads assets to R2
- Stores embeddings in `pgvector`
- Completes the run, broadcasts realtime

12

Forge Integration

NEW IN v1.3

Forge is the visual front door to the kernel. Its node editor does not run anything itself — it **compiles** a visual graph into the exact same Blueprint + Execution Contract artifacts the kernel already governs. The no-code surface and the deterministic kernel share one contract format.

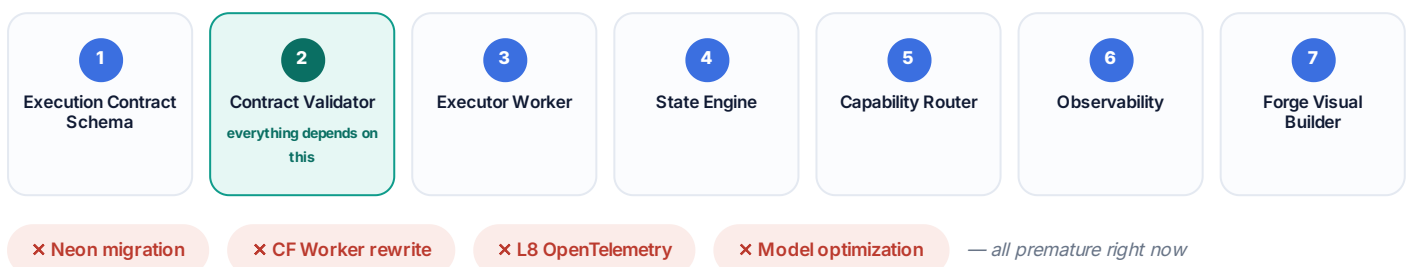


- ✓ Each Forge node maps to one declared capability with its allowed tools, expected output, and SLA — the node *is* a contract fragment.
- ✓ Publishing in Forge writes a new `blueprint_versions` row and runs `blueprint_tests` before `blueprint_status` can become `active`.
- ✓ This is the bridge from Forge → full no-code platform → AI Co-Founder: the same contract format scales from hand-written JSON to a visual builder without a second runtime.

13

Build Priority & The Funding Thesis

The reviewers were explicit about what *not* to start with: not the Router, not the Neon migration, not model optimization. The whole kernel depends on the Execution Contract Validator, so it comes first.



Fundable systems are proven by receipts, not diagrams.

Input → Factory → Artifact → Evidence → Decision → Receipt

One vertical slice that actually ran — gated by Mac Law, with auditable receipts — is Proof of Deterministic Execution. That is the demo that wins funding, and it outweighs a hundred pages of architecture.

13.1 The One Truth Ticket

- 1 Pick **one** factory (video-ad or forge drain).
- 2 Run it end-to-end through SCAN→SHIP on the existing Railway FBE.
- 3 Emit receipts shaped exactly as `artifact` / `evidence` / `decision` — JSON now, SQL later.
- 4 Gate every advance on a validator/Hub receipt, never agent self-report.
- 5 Package the run as a one-screen investor demo.

SOURCE-A CLOUD KERNEL v1.3 — TARGET ARCHITECTURE

This is the target vision for the Source-A cloud kernel. It is authoritative for *where we are going*, not for what runs today. Current runtime truth lives in the L0 Mac Workbench; the two are reconciled by the SSOT Index.

THE GOVERNING PRINCIPLE

Contracts run the system.

LLMs only propose plans. The kernel holds execution authority.

Noetfield Systems Inc. | TrustField Technologies Inc.

Vancouver, BC — 2026 · External review: 9.3 / 10